

Week 04 - Function Calling con bucle (wget + SQLite)

0. Contexto y objetivo

En esta practica, como grupo, implementamos un programa en Python donde el LLM puede usar 2 herramientas:

- `wget`: descarga contenido de una URL (con confirmación humana)
- `execute_sql`: ejecuta SQL sobre SQLite

La parte clave era hacerlo en bucle de rondas, no en una sola llamada.

1. Step 0 - Research previo

1.1 SQLite y sqlite3

Comprobamos en CLI:

- crear tabla
- insertar filas
- seleccionar datos
- ver `.schema` y `.tables`

Lo importante que vimos: SQLite no es un servidor, es un archivo `.db`. Para esta practica viene bien por simplicidad, aunque para escenarios grandes o mucha concurrencia tendria mas sentido una BD de servidor.

Evidencia:

- `evidence/step0_research.txt`

1.2 `subprocess.run()`

Probamos un comando con `sleep` para verificar comportamiento.

Conclusion: `subprocess.run()` bloquea hasta que acaba (sincronizo).

Tambien repasamos:

- `capture_output=True` para capturar salida
- `text=True` para trabajar con strings
- `timeout` para evitar bloqueos largos

Evidencia:

- `evidence/step0_research.txt`

1.3 `wget`

Probamos:

- URL valida
- URL no existente

Con URL invalida salida codigo de error (exit code 4), y eso nos servirá luego para Step 5.

Evidencia:

- `evidence/step0_research.txt`
-

2. Step 1 - Setup y una tool call

Estructura que dejamos en el proyecto:

- `tool_loop_agent.py`
- `.env.example`
- `.gitignore`
- `pyproject.toml`

Schema de `execute_sql` (fragmento):

```
{
  "type": "function",
  "function": {
    "name": "execute_sql",
    "description": "Run SQL against local SQLite database.db.",
    "parameters": {
      "type": "object",
      "properties": {
        "query": {"type": "string"}
      },
      "required": ["query"],
    },
  },
}
```

Prueba del paso:

- prompt para crear tabla test
- el modelo pidió `execute_sql`
- se ejecuta correctamente

Evidencia:

- `evidence/step1_single_tool_call.txt`
-

3. Step 2 - Loop con varias tool calls

Implementamos el patrón de loop: llamar al modelo, ejecutar tools, devolver resultados, repetir.

Fragmento del bucle:

```
if not message.tool_calls:
    final_text = message.content or ""
    return final_text

messages.append({"role": "assistant", "tool_calls": [...]}))
...
messages.append({
    "role": "tool",
    "tool_call_id": tc.id,
    "name": tool_name,
    "content": tool_result,
})
```

Prueba del paso:

- prompt: crear tabla cities e insertar 3 ciudades
- se observaron llamadas secuenciales de `execute_sql`

Evidencia:

- `evidence/step2_loop_sequence.txt`

Como sabe el programa cuándo parar: cuando la respuesta ya no trae `tool_calls`.

4. Step 3 - Tool `wget` con confirmación humana

Schema de `wget` (fragmento):

```
{
  "type": "function",
  "function": {
    "name": "wget",
    "description": "Fetch content from a URL using system wget. This tool requires expli",
    "parameters": {
      "type": "object",
      "properties": {
        "url": {"type": "string"},
        "flags": {"type": "string"},
      },
      "required": ["url"],
    },
  },
}
```

```
}
```

Logica de confirmacion (fragmento):

```
print(f"[confirm] LLM wants to run: {command_text}")
answer = input("Allow command? (y/n): ").strip().lower()
if answer != "y":
    return json_result(False, error="USER DENIED: command was not executed.")
```

Probamos los 2 escenarios pedidos:

1. aprobación de comando
2. denegación de comando

Evidencia:

- evidence/step3_wget_approve.txt
 - evidence/step3_wget_deny.txt
-

5. Step 4 - Test completo (fetch + store + query)

Prompt usado:

Fetch <https://jsonplaceholder.typicode.com/users> and store the id, name, email, and city of every user in a SQLite table called users. Show me the final contents of the table.

Flujo observado en nuestra ejecución principal:

1. wget
2. execute_sql para CREATE TABLE
3. execute_sql para INSERT
4. execute_sql para SELECT
5. respuesta final del asistente

Iteraciones del loop en este test: 5

Evidencias:

- salida principal: evidence/step4_full_test.txt
- verificación independiente: evidence/step4_sqlite_verify.txt

La verificación con sqlite3 devolvió las 10 filas esperadas.

6. Step 5 - Salida legible y manejo de errores

Mejoramos logs para mostrar:

- nombre del tool
- argumentos

- preview del resultado

Y protegimos ejecución con try/except para evitar cada del programa.

Errores probados:

1. URL invalida
2. SQL invalido (tabla inexistente)
3. denegación de wget

Evidencias:

- evidence/step5_bad_url.txt
- evidence/step5_invalid_sql.txt
- evidence/step3_wget_deny.txt

En todos los casos el programa siguió funcionando y devolvió error controlado al modelo.

7. Preguntas obligatorias

1) How does your program know when to stop calling the LLM?

Para cuando message.tool_calls está vacío. En ese punto se toma message.content como respuesta final.

2) What is the role of tool_call_id in the message protocol?

Permite enlazar cada mensaje role=tool con la llamada exacta que pidió el asistente. Sin eso, la conversación de tools se descoordina.

3) Why is user confirmation important for the wget tool but not for execute_sql?

wget toca red y recursos externos, por eso requiere validacion humana. execute_sql en este ejercicio está limitado a una BD local controlada.

4) What happens in the conversation when the user denies a wget command?

Se devuelve un resultado de tool tipo USER DENIED... , se añade al historial, y el modelo responde adaptandose a esa denegación.

5) How many iterations did the loop run for the full test prompt? Were you surprised?

En nuestra ejecución principal fueron 5 iteraciones. No nos sorprendió, era el número esperado para ese flujo.

8. Extra challenge (opcional) - ¿Importa el modelo?

Probamos también `qwen2.5-v1-72b-instruct` con el mismo prompt. Observamos más iteraciones y más errores de formato en argumentos de tool.

Evidencia:

- `evidence/extra_model_compare_qwen2_5_v1_72b.txt`

Conclusión: sí importa, bastante, para function calling robusto.

9. Seguridad de API key

Verificado en el proyecto:

- `.env` no se sube
 - `.env.example` incluido
 - `.gitignore` contiene:
 - `.env`
 - `database.db`
 - `__pycache__`
 - `.venv`
-
-

11. Anexo A - Salida completa Step 4 (raw)

Week-04 Tool Loop Agent

Model: `qwen3.5-122b-a10b`

Endpoint: `https://dashscope-intl.aliyuncs.com/compatible-mode/v1`

Database: `/tmp/week04/database.db`

Confirm policy: `always`

Prompt: `Fetch https://jsonplaceholder.typicode.com/users and store the id, name, email, and`

=== Iteration 1 ===

[assistant] Requested 1 tool call(s)

[tool-call] `id=call_939f4e38614b401ba2a4f6c3 name=wget`

[tool-args] `{"url": "https://jsonplaceholder.typicode.com/users"}`

[confirm] LLM wants to run: `wget -q -O - https://jsonplaceholder.typicode.com/users`

[confirm] Auto-approval policy active: `yes`

[tool-result-preview] `{"ok": true, "data": {"command": "wget -q -O - https://jsonplaceholder`

=== Iteration 2 ===

[assistant] Requested 1 tool call(s)

[tool-call] `id=call_be177700108a4924877055b4 name=execute_sql`

[tool-args] `{"query": "CREATE TABLE IF NOT EXISTS users (id INTEGER PRIMARY KEY, name TEXT,`

```
[tool-result-preview] {"ok": true, "data": {"rowcount": -1, "lastrowid": 0}, "error": null}
```

```
=== Iteration 3 ===
```

```
[assistant] Requested 1 tool call(s)
```

```
[tool-call] id=call_2e3c409974a744bf88ce4616 name=execute_sql
```

```
[tool-args] {"query": "INSERT INTO users (id, name, email, city) VALUES (1, 'Leanne Graham'}
```

```
[tool-result-preview] {"ok": true, "data": {"rowcount": 10, "lastrowid": 10}, "error": null}
```

```
=== Iteration 4 ===
```

```
[assistant] Requested 1 tool call(s)
```

```
[tool-call] id=call_d0113c8b1b6648f49113563e name=execute_sql
```

```
[tool-args] {"query": "SELECT * FROM users"}
```

```
[tool-result-preview] {"ok": true, "data": {"columns": ["id", "name", "email", "city"], "row
```

```
=== Iteration 5 ===
```

```
[assistant-final]
```

```
I have successfully fetched the user data from the JSONPlaceholder API, created a SQLite table
```

```
Here are the final contents of the `users` table:
```

```
| id | name | email | city |
| :--- | :--- | :--- | :--- |
| 1 | Leanne Graham | Sincere@april.biz | Gwenborough |
| 2 | Ervin Howell | Shanna@melissa.tv | Wisokyburgh |
| 3 | Clementine Bauch | Nathan@yesenia.net | McKenziehaven |
| 4 | Patricia Lebsack | Julianne.OConner@kory.org | South Elvis |
| 5 | Chelsey Dietrich | Lucio_Hettinger@annie.ca | Roscoeview |
| 6 | Mrs. Dennis Schulist | Karley_Dach@jasper.info | South Christy |
| 7 | Kurtis Weissnat | Telly.Hoeger@billy.biz | Howemouth |
| 8 | Nicholas Runolfsdottir V | Sherwood@rosamond.me | Aliyaview |
| 9 | Glenna Reichert | Chaim_McDermott@dana.io | Bartholomebury |
| 10 | Clementina DuBuque | Rey.Padberg@karina.biz | Lebsackbury |
```

```
[summary] iterations=5 final_text_preview=I have successfully fetched the user data from the
```

12. Anexo B - Verificación SQLite independiente (raw)

```
1|Leanne Graham|Sincere@april.biz|Gwenborough
2|Ervin Howell|Shanna@melissa.tv|Wisokyburgh
3|Clementine Bauch|Nathan@yesenia.net|McKenziehaven
4|Patricia Lebsack|Julianne.OConner@kory.org|South Elvis
5|Chelsey Dietrich|Lucio_Hettinger@annie.ca|Roscoeview
6|Mrs. Dennis Schulist|Karley_Dach@jasper.info|South Christy
7|Kurtis Weissnat|Telly.Hoeger@billy.biz|Howemouth
8|Nicholas Runolfsdottir V|Sherwood@rosamond.me|Aliyaview
9|Glenna Reichert|Chaim_McDermott@dana.io|Bartholomebury
```

10|Clementina DuBuque|Rey.Padberg@karina.biz|Lebsackbury
